

Subgraph & Gas-Benchmark Audit

Correctness of the indexed gas benchmarks behind the private-transfers evaluations

Target: `private-transfers-benchmarks` — `subgraph/`, `gas-benchmarks/`, `project-evaluations/scripts/sync-benchmarks/`

Branch: `subgraph-review` · Commit: `38bce8d` · Date: 2026-06-25

Privacy Stewards of Ethereum — Private Transfers Engineering

Executive summary

The pipeline surfaces several wrong published costs, but the severities below route through one test: does the defect change the dollar figure a user sees? Most do not.

The original complaint — Privacy Pools deposit at ~\$2.40, roughly 11× its peers — is real and live, but it is a corrupted aggregate in the deployed subgraph rather than a handler bug: the same `handleDeposit` reproduces ~410k gas for the 13 other shield pools, and only pool `0xf241d57c` reads 4,614,983. A clean re-index fixes it (Critical 1).

Two code defects move a surfaced number. Railgun counts commitments and output-notes instead of transactions, so its transfer cost reads 672k against a measured ~1.06M per pure transfer — ~1.6× understated (High 1). And an unguarded NaN in the sync layer throws and aborts the whole multi-protocol run; Worm withdraw, which records no gas by construction, is the live trigger and currently ships a fabricated `$0.00` (High 2).

The review lists 24 findings — 1 critical (operational), 2 high, 1 medium, 9 low, 9 informational, plus 1 inconclusive and 1 refuted — each confirmed in code at `file:line` and, for every surfaced value, reconciled against on-chain gas reproduced over fixed block ranges (cross-check and Appendix B). Severities were re-graded after an independent adversarial re-review to track surfaced-cost impact, and shared root causes were merged.

One result is worth stating plainly because it overturns an intuitive theory: crediting whole-transaction gas to each event does not, on its own, inflate the average. Where a handler increments count alongside gas the two scale together and the per-operation average survives (Hinkal, Low 2); the damage there is to protocol totals and per-transaction records, not to the surfaced cost.

Outside the findings above, every surfaced cost matches the chain. The fixes are mostly small and local.

Scope and target

Reviewed at commit `38bce8d` on branch `subgraph-review`, across three areas:

- **Subgraph mappings** — `subgraph/src/**`: the AssemblyScript event/call handlers for 14 protocols (railgun, tornado-cash, hinkal, privacy-pools, zerc20, intmax, worm, blanksquare, veil-cash, curvy, redact, fluidkey, monero, houdiniswap), their per-protocol `utils.ts`, the merged `schema.graphql`, and the manifests under `subgraph/configuration/`.
- **Benchmark pipeline** — `gas-benchmarks/src/**`: the `SubgraphService` and per-network GraphQL fragments that query the deployed subgraphs and write `benchmarks.json`, plus the hardcoded-constant protocols (curvy, fluidkey, worm, houdiniswap, monero).
- **Sync layer** — `project-evaluations/scripts/sync-benchmarks/**`: `index.ts` (per-protocol mapping), `utils.ts` (`syncOperation`, `buildCostValueAndNotes`), and `interfaces.ts`.

Out of scope: the on-chain contracts, the evaluation content, and the frontend that renders the values.

System overview

The pipeline turns indexed events into a USD cost per private-transfer operation, in three stages.

A **subgraph** indexes each protocol's contract events. A handler runs once per matching log and writes two kinds of entity: an immutable per-operation record (`id = txHash-logIndex`, carrying `gasUsed`, `gasPrice`, `value`) and a set of mutable running aggregates updated with `.plus()` — a protocol-level total, a per-operation total (`shield / unshield / transact`), and per-token totals. Gas is read from `event.receipt.gasUsed`, which requires `receipt: true` in the manifest. Worm is the exception: it uses a call handler, which has no receipt.

The **benchmark pipeline** (`gas-benchmarks`) queries each deployed subgraph for those aggregates and writes `benchmarks.json`. A few protocols that are not subgraph-indexed (Curvy, Fluidkey, Worm, Houdiniswap, Monero) instead contribute hardcoded or externally-fetched figures here.

The **sync layer** reads `benchmarks.json` and, per protocol, picks the operation or token stat that represents deposit / transfer / withdraw, computes `average = Number(totalGasUsed) / Number(totalCount)` in `syncOperation`, and converts it to "`~X ETH / $Y`" via `buildCostValueAndNotes` using a live gas price and ETH/USD rate. That string is written into the protocol's evaluation JSON. The whole audit concerns whether the number at the end of this chain reflects the real per-operation gas.

Methodology

The review ran as a structured multi-agent engine, deliberately built so no finding rests on a vote — the first-round single panel reached a wrong consensus, and only measurement corrected it.

1. **Six independent auditors**, each reviewing the entire codebase (not slices) through a distinct lens: a The Graph / graph-node indexer specialist, a gas/economics analyst, a data-pipeline reviewer, a backend-TypeScript reviewer, an adversarial falsifier, and a lead. None saw another's output. They produced 77 raw findings.
2. **Consolidation** into 28 canonical findings, keyed by file and symbol, merging duplicates and keeping genuinely distinct issues apart.
3. **Per-finding verification**. Each candidate went to an independent verifier that read the cited code and returned `confirmed`, `refuted`, or `inconclusive`, with `file:line` evidence, a quoted snippet, and a concrete fix. A finding is reported as confirmed only if the verifier confirmed it against the source. Because these are TypeScript / AssemblyScript defects, verification is in code.
4. **On-chain reconciliation** of the surfaced values, separate from code review: the deployed subgraphs were queried directly for their aggregates, and the on-chain side was measured from sampled transactions over fixed block ranges (cross-check section and Appendix B).

5. **Scoring and re-grade.** The panel scored every confirmed finding; an independent adversarial re-review then re-graded severities to surfaced-cost impact and merged shared root causes. Appendix A maps each original finding to its final disposition.

What we did not do

- Tornado Cash was not measured on-chain: its instance contracts are templated and the relevant addresses are sanctioned, so public RPCs refuse them. Its subgraph aggregates were read directly.
- Worm and Monero have no measurable on-chain event for this purpose (a call handler and a non-EVM chain respectively).
- Reproduction was run against Alchemy RPCs and the Graph Studio endpoints supplied by the team.

Independent re-review

This report was then re-audited adversarially: every code claim re-checked at `file:line`, every surfaced magnitude reproduced live against the deployed subgraphs and RPCs (Appendix B), severities re-graded to surfaced-cost impact, and shared root causes merged. The findings and severities below reflect that re-grade; Appendix A maps each original finding to its final disposition.

Severity and status

Severity is calibrated to impact on a published number, not exploitability:

- **Critical** — corrupts a headline published number badly, or breaks the sync run.
- **High** — corrupts or fabricates a surfaced cost for a protocol.
- **Medium** — corrupts stored data or a value field, or is a real defect not currently surfaced.
- **Low / Informational** — latent, cosmetic, or precision-only.

Findings are identified by severity and an index within it (Critical 1, High 2, ...). Confirmation status is **Confirmed**, **Inconclusive**, or **Refuted**. Every confirmed finding is **Open** (unremediated) at commit `38bce8d`.

Findings summary

ID	Title	Severity	Status
Critical 1	Privacy Pools deposit aggregate inflated ~11x in the deployed index (operational)	Critical	Confirmed
High 1	Railgun aggregates count commitments and output-notes, not transactions	High	Confirmed
High 2	Unguarded NaN in the sync layer aborts the whole run; Worm withdraw is the trigger	High	Confirmed
Medium 1	Intmax Scroll withdraw reported per-withdrawal, ~6.7x below the per-tx basis peers use	Medium	Confirmed
Low 1	Monero transfer surfaces a raw XMR fee (or "undefined") in the ETH/\$ column	Low	Confirmed
Low 2	Hinkal per-log handlers double-count protocol totals (surfaced average survives)	Low	Confirmed
Low 3	Veil withdrawal can be misclassified as transfer on a preceding 2-topic log (latent)	Low	Confirmed
Low 4	Privacy Pools shield keyed on pool address, unshield on asset address	Low	Confirmed
Low 5	Tornado withdrawal totalValue uses tx.value (~0 for relayed withdrawals)	Low	Confirmed
Low 6	Note-merge via <code>split('.')</code> drops authored note content	Low	Confirmed
Low 7	Privacy Pools deposit updates stats before the pool-null return	Low	Confirmed
Low 8	Tornado/Intmax resolve token/amount via in-handler <code>eth*call</code> (no <code>try*</code>)	Low	Confirmed
Low 9	Curvy deposit and transfer legs priced against different reference networks	Low	Confirmed
Informational 1	Hardcoded 21,000-gas constant surfaced as a "Computed average" (value correct)	Informational	Confirmed
Informational 2	Intmax deposit surfaces a blended all-token average, not ETH-specific	Informational	Confirmed
Informational 3	Veil withdrawal value decoded little-endian (value never consumed)	Informational	Confirmed
Informational 4	Redact/Veil totalValue never populated	Informational	Confirmed
Informational 5	Curvy/Fluidkey accumulate one operation into two entities sharing an id	Informational	Confirmed
Informational 6	zERC20/Redact re-load + save protocol stats (latent lost update)	Informational	Confirmed

ID	Title	Severity	Status
Informational 7	Null-receipt skew, latent under <code>receipt: true</code> manifests	Informational	Confirmed
Informational 8	<code>buildCostValueAndNotes</code> truncates the average with <code>Math.trunc</code>	Informational	Confirmed
Informational 9	<code>syncOperation</code> averages with <code>Number()</code> on $>2^{53}$ -capable sums	Informational	Confirmed
Inconclusive 1	Worm tx-hash dedup may drop a second swap in one transaction	Low	Inconclusive
Refuted 1	SubgraphService shares one cache file across five network queries	n/a (refuted)	Refuted

Detailed findings

Every finding is **Open** (unremediated) at commit `38bce8d`. Locations are `file:line` against that commit.

Critical 1 — Privacy Pools deposit aggregate inflated ~11x in the deployed index (operational)

The deployed subgraph's `privacy-pools-shield-0xf241d57c...` entity holds `totalGasUsed = 20,054,718,016` over `totalCount = 4,346` — an average of 4,614,983 gas, surfaced as ~\$2.40. On chain, deposits to that exact pool cost ~410k and emit one `Deposited` event per transaction; the reproduction measures 418,187 (Appendix B). The stored aggregate is ~11.0x too high.

This is not a code defect. `handleDeposit` reads `event.receipt!.gasUsed` and increments count and gas together (`subgraph/src/privacy-pools/privacy-pools.ts:110-160`), so the handler reproduces ~410k — and does for every other pool: the control pool `0xb419` reads `276,566,142 / 648 = 426,800`, matching chain, while only `0xf241` reads 4,614,983 under the same code. The corruption is in the deployed index — a replay/re-index/redeploy double-count of the mutable accumulator while the immutable per-deposit entities stay correct.

Impact. The headline cost a reader compares against peers is ~11x too high, live. Only this pool's aggregate is affected. This is an operational/deployment finding, not a code fix.

Recommendation. Re-index the mainnet subgraph from `startBlock`, then re-run `pnpm run benchmark` and `pnpm run sync:benchmarks`; the deposit average should fall to ~410k. Structurally, deriving aggregates from the immutable per-deposit entities removes this failure mode (see Low 2).

High 1 — Railgun aggregates count commitments and output-notes, not transactions

Railgun's handlers count commitments and output-notes instead of transactions, and the shield token row pre-divides gas per commitment — one mechanism with one fix, surfaced on two operations.

- **Transfer (the surfaced error).** `handleTransact` increments `transactStats.totalCount` by `event.params.hash.length` (the output-commitment array) while adding the full receipt gas once (`subgraph/src/railgun/railgun-smart-wallet.ts:227,230`); `sync` surfaces this operation stat (`project-evaluations/scripts/sync-benchmarks/index.ts:145`) and `syncOperation` divides `totalGasUsed/totalCount` (`utils.ts:152`), giving `whole-tx-gas ÷ output-commitments`. Reproduced: the deployed aggregate is $114,959,451,261 / 170,995 = 672,297$ against a measured 1,062,775 per pure-transfer transaction (Appendix B, unshield-excluded) — **1.58× understated**.
- **Deposit (latent).** `handleShield` adds `gasPerCommitment = receipt.gasUsed.div(commitments.length)` and `totalCount += 1` per commitment to the token row `sync` reads (`railgun-smart-wallet.ts:135-140` ; `index.ts:136-143`). With shields running ≈ 1 commitment today the divisor is 1, so the figure is correct now (WETH aggregate $845,866 \approx$ measured $756,217$); it understates by $\sim 1/K$ once K-commitment shields appear.
- **Non-surfaced.** The protocol/operation counts mix `commitments.length`, `hash.length` and `+1` (`railgun-smart-wallet.ts:94-110,156,218-234`), so `totalTxCount` is a commitment count for shields; and the per-commitment `BigInt.div` at `:135` drops up to N-1 gas units per batch. Neither is read for a surfaced cost.

Recommendation. Count one transaction per shield/unshield/transact event (`totalCount += 1`) and keep the full receipt gas, matching the per-transaction divisor used everywhere else. Keep per-commitment figures only on the per-token rows that need them.

High 2 — Unguarded NaN in the sync layer aborts the whole run; Worm withdraw is the live trigger

`syncOperation` computes `average = Number(operation.totalGasUsed) / Number(operation.totalCount)` with no guard for a zero count or a missing operand (`project-evaluations/scripts/sync-benchmarks/utils.ts:152`); `buildCostValueAndNotes` then runs `BigInt(Math.trunc(average))`, which throws `RangeError` on NaN (verified in node). Because the callers run inside `Promise.all` (`index.ts:36`), one throw aborts every protocol's sync, and the `find(...)! assertions` in `index.ts` can feed undefined straight in.

Worm withdraw is the live instance. It is indexed by a call handler (`SwapBethWithEthCall`) with no receipt, so `WormOperationStats` only ever sets `totalCount`, never `totalGasUsed`, and neither the schema nor the fragment has the field (`subgraph/src/worm/beth-to-eth.ts:9-19,63-65` ; `gas-benchmarks/src/subgraph/worm.ts:9-14`); `interfaces.ts:56-58` wrongly marks `totalGasUsed` required, so `tsc` never flags the gap. The sync divides `Number(undefined)/N = NaN` and throws. The

committed `worm.json` carries a stale `~0.000000` ETH / `$0.00` that the next run cannot correct because it crashes first.

Impact. Run-breaking: one bad operation stat aborts the multi-protocol sync and leaves every evaluation un-updated. Worm's published withdraw cost is a fabricated zero, not a measurement — and on a user-paid-gas basis a relay-broadcast ZK mint costs the user ≈ 0 ETH gas, so there is no plausible non-zero figure to surface.

Recommendation. Guard `syncOperation` (skip or write "N/A" when the stat is missing or the count is zero; assert `Number.isFinite(average)` before formatting) and replace the `find(...)`! assertions with explicit checks. Stop routing Worm withdraw through `syncOperation` — special-case it like the Worm deposit estimate, or move it to an event handler that exposes a receipt and add `totalGasUsed` to the entity and both fragments. Fix the interface so the absent field is visible to `tsc`.

Medium 1 — Intmax Scroll withdraw is reported per-withdrawal, $\sim 6.7\times$ below the per-transaction basis peers use

`handleWithdrawal` runs once per `DirectWithdrawalQueued` log and divides receipt gas by the count of those logs in the transaction (`subgraph/src/intmax/withdrawal.ts:78,96,109`), so the surfaced operation figure (`project-evaluations/scripts/sync-benchmarks/index.ts:108`) is gas-per-withdrawal: 117,857 against a measured per-transaction 779,990 at 6.748 events/tx (Appendix B). Every peer protocol reports per-transaction, so on the shared column Intmax withdraw reads $\sim 6.7\times$ cheaper. The basis is defensible — Intmax batches many users' withdrawals into one operator transaction, so amortising across users is arguably the truer per-user cost — but it breaks comparability with the rest of the table, and it is internally inconsistent: the protocol-level totals add the full receipt gas per event (`withdrawal.ts:62,65`) while the operation, entity, and token rows divide. Those protocol totals are not surfaced.

Recommendation. Pick one basis and apply it across all four levels. For per-transaction parity with peers, drop the `.div(totalWithdrawals)` and dedup by transaction hash; if per-withdrawal is intended, document it in the property note and amortise the protocol totals the same way.

Low and informational

- **Low 1 — Monero transfer surfaces a raw XMR fee in the ETH/\$ column** (`project-evaluations/scripts/sync-benchmarks/index.ts:114–116`). `String(transfer.averageTxFee)` writes an average in whole XMR (`gas-benchmarks/src/monero/utils.ts:198–205`), never gas, never USD-converted, while every other protocol writes "`~X ETH / $Y`"; if `getMoneroMetrics` returns `{}` the value becomes the literal `"undefined"`. The note discloses XMR and the absolute is $\sim \$0.04$, so the live harm is small — but an XMR fee is not comparable to EVM gas at all, so the better fix may be to drop Monero from the surfaced table rather than reformat the cell.
- **Low 2 — Hinkal per-log handlers double-count protocol totals and overwrite entities** (`subgraph/src/hinkal/hinkal-pool.ts:156–357`). Each handler fires once per log and inspects the whole receipt, so for a transaction with `N` matching logs the protocol totals add the full

receipt gas N times and the per-tx entity (keyed on tx hash, no log index) is overwritten. The surfaced **average** survives because count scales with gas ($\approx 1.05M \approx$ measured 1.076M); the damage is to non-surfaced protocol totals and lost per-tx records. Dedup on the per-tx entity before mutating stats.

- **Low 3 — Veil withdrawal can be misclassified as transfer** (`subgraph/src/veil-cash/utils.ts:6-20`). `getWithdrawalEvent` returns `null` on the first 2-topic log whose `topic[0]` is not the Withdrawal signature, instead of scanning on, so an unrelated 2-topic log before the real WETH Withdrawal shifts count and gas from withdraw to transfer. Latent — Veil withdraw matches chain today (1.30M vs 1.31M), so it is not firing. Change the inner `return null` to `continue`.
- **Low 4 — Privacy Pools shield/unshield keyed inconsistently** (`subgraph/src/privacy-pools/privacy-pools.ts:50-65, 137, 151, 208`). Shield stats are keyed on `event.params._pool` (a field named `tokenAddress`) while unshield uses `event.params._asset`; sync matches shield by a pool-address constant and unshield by the native-ETH asset constant. It resolves correctly today only because the constants were hand-picked to the two keyings, with no runtime guard behind the `find(...)`!. Key both on `pool.asset` and match against one asset constant.
- **Low 5 — Tornado withdrawal `totalValue` uses `tx.value`** (`subgraph/src/tornado-cash/tornado-instance.ts:113, 142`). The entity sets `amount = denomination` correctly, but the aggregate accumulates `event.transaction.value` (~ 0 for relayed withdrawals). The value is consumed downstream (`gas-benchmarks/src/subgraph/tornado-cash.ts:17-22`, persisted at `gas-benchmarks/src/subgraph/index.ts:50`) but feeds no surfaced cost; the matchstick test masks it because the mock's default `tx.value` of 1 coincides with the mocked denomination 1. Accumulate `denomination`.
- **Low 6 — Note-merge drops authored content** (`project-evaluations/scripts/sync-benchmarks/utils.ts:122-127`). When a note already carries the "Computed average..." prefix, the human note is recovered via `existing.notes.split(' ')[1]`, keeping only the segment between the first and second `.`. Multi-sentence notes lose everything after the second separator — on the second sync onward, then stable. Surfaced value untouched. Strip the known prefix with an anchored regex instead.
- **Low 7 — Privacy Pools deposit mutates stats before the `pool === null` return** (`subgraph/src/privacy-pools/privacy-pools.ts:110-135`), so a deposit whose pool entity is not yet indexed inflates operation totals but is invisible to the per-token metric. Move the pool lookup and guard above the mutations.
- **Low 8 — Tornado/Intmax resolve contract data via in-handler `eth_call` without the `try_` variant** (`subgraph/src/intmax/liquidity-v2.ts:85-87`, `tornado-instance.ts:106-107`), so a revert aborts the handler. Intmax's `getTokenInfo` resolves only the token address (the amount comes from `event.params.amount`); only Tornado's `denomination()` resolves an amount. Latent under fixed-block replay. Switch to `try_` or resolve from registration-time data.
- **Low 9 — Curvy deposit and transfer legs priced against different reference networks** (`project-evaluations/scripts/sync-benchmarks/index.ts:64-72`, default at `utils.ts:92`). The transfer syncs with `Network.ARBITRUM`; the deposit omits the network argument and defaults to `Network.MAINNET`. The 21,000-gas count is network-independent, only the fiat reference differs, and the live effect is $\sim \$0.01$ — but the mainnet default is an unmarked parameter, not a deliberate convention. Price both legs against the same network or document it.

- **Informational 1 — Hardcoded 21,000-gas constant surfaced as a "Computed average"** (`gas-benchmarks/src/utils/constants.ts:52`; `Curvy/Fluidkey/Worm` deposit, `Houdiniswap` transfer, set in the per-protocol `index.ts`). `buildCostValueAndNotes` stamps the same note as measured averages, so a reader cannot tell the guess from a measurement. The figure is right — these ops are native-ETH transfers — so the defect is provenance/labelling only. Emit an "Estimated..." note for the constant-fed call sites.
- **Informational 2 — Intmax deposit surfaces a blended all-token average** (`project-evaluations/scripts/sync-benchmarks/index.ts:104`). `Sync` passes the operation-level deposit stats (all tokens) instead of filtering to the native-ETH per-token entity peers use. Impact 0.045% today (blended 200,659 vs native 200,569). Filter to native ETH for consistency.
- **Informational 3 — Veil withdrawal value decoded little-endian** (`subgraph/src/veil-cash/veil-eth-pool.ts:66`). `BigInt.fromUnsignedBytes` reads the big-endian ABI word as little-endian, so the stored value is byte-reversed. Gas is unaffected and the value is never queried or consumed. Decode with `ethereum.decode("uint256", data).toBigInt()`, matching the Hinkal pattern.
- **Informational 4 — Redact/Veil totalValue never populated** (`subgraph/src/redact/confidential-eth.ts:162-165`). A `TODO` leaves it at 0 because the `Unshielded` event's `amount` is an FHE ciphertext (an `euint64` handle stored as `bytes32`), so `.plus(amount)` is meaningless — the value must come from the paired `Transfer()` event. The same shape exists in `VeilCashTransferStats.totalValue`; a private transfer has no public value, so 0 is defensible.
- **Informational 5 — Curvy/Fluidkey accumulate one operation into two entities sharing an id** (`subgraph/src/curvy/curvy-vault-v6.ts`, `subgraph/src/fluidkey/smart-account-relayer.ts`), leaving a duplicate counter the `sync` never reads — both entities carry identical totals, so the surfaced ratio is invariant. Drop the duplicate or give the child a distinct id.
- **Informational 6 — zERC20/Redact re-load and save protocol stats** (`subgraph/src/zerc20/zerc20.ts:30-35`, `confidential-eth.ts:109-153`). The handler loads a read-only handle, then `saveProtocolStats` re-loads and saves independently. No fault today; latent lost-update if a future edit saved the stale first handle. Pass the loaded handle in.
- **Informational 7 — Null-receipt skew, latent under receipt: true manifests.** Most handlers increment `totalCount` unconditionally but `totalGasUsed` only inside `if (event.receipt !== null)`, while `Hinkal/Veil` early-return — an inconsistency that would bias the average down on a null receipt. Every cited handler declares `receipt: true` and graph-node attaches a receipt when it is set, so the branch is unreachable in production. Adopt one convention so it cannot drift.
- **Informational 8 — buildCostValueAndNotes truncates the average with Math.trunc** before pricing (`project-evaluations/scripts/sync-benchmarks/utils.ts:94`), a sub-gas-unit systematic downward bias. Use `Math.round`.
- **Informational 9 — syncOperation averages with Number()** on totals that could exceed 2^{53} (`utils.ts:152`). Latent only ($\sim 2^{35}$ today); divide in `BigInt` if totals could ever grow that large.

Note (not a finding) — Veil deposit surfaces the user queue leg only (`project-evaluations/scripts/sync-benchmarks/index.ts:162-167`). The deposit is a two-transaction flow: the user's `DepositQueued` leg (the surfaced $\sim 360k$) and an operator-submitted `DepositAccepted` leg

(~1.36M). The accept gas is paid by the operator, not the depositor (`DepositAccepted` carries an indexed `operator` param), so for a user-paid-gas metric excluding it is correct by design. Document the choice in the property note so the figure is not read as end-to-end deposit cost.

Inconclusive and refuted

- **Inconclusive 1 — Worm tx-hash dedup may drop a second swap in one transaction** (`beth-to-eth.ts:38-44`). The code keys `WormWithdraw` by transaction hash and early-returns, so a second `SwapBethWithEth` call in one transaction is dropped. `worm/README.md:34` does not justify this — it describes a different, unimplemented mechanism (scanning WETH `Withdraw` events to dedupe fee-hook legs), while the shipped code is a `swapBethWithEth` call handler keyed on tx hash. It is a defect only if two genuinely independent user withdrawals can share one transaction, which the code alone cannot establish. Confirm whether the `BETHToETH` contract can invoke `swapBethWithEth` twice with distinct recipients in one transaction; if so, key by tx hash plus call index.
- **Refuted 1 — SubgraphService shares one cache file across five network queries** (`subgraph/index.ts`). The headline claim was cross-network cache corruption. The decorator keys the cache by method name and args (`cache.ts:53`), so the five networks occupy distinct keys and cannot collide, and each entry is typed correctly via the decorator's generic. The only true sub-claim is that the in-memory `ICacheValue.data` type is wrong, but that map is never read (write-only dead state), so it has no functional effect.

Gas cross-check

"Subgraph gas" is the deployed subgraph's own average (`totalGasUsed / totalCount`) for the exact stat the sync surfaces, queried live from the Graph Studio endpoints. "Measured gas/tx" is the mean of five sampled transactions over fixed block ranges (Appendix B). Δ is `subgraph ÷ measured`.

Protocol	Operation	Subgraph gas	Measured gas/tx	Δ	Surface d cost	Note
Privacy Pools	deposit (pool 0xf241)	4,619,362	418,187	11.0x	\$2.40	Critical 1
Privacy Pools	deposit (control 0xb419)	426,715	441,217	0.97x	not surfaced	control pool, shown for comparison
Privacy Pools	withdraw (ETH)	607,484	609,544	1.0x	\$0.31	consistent
Railgun	deposit (shield)	891,997	756,217	1.2x	\$0.44	consistent ($\approx$ 1 commitment)
Railgun	transfer (transact)	672,318	1,062,775	0.63x	\$0.35	High 1 — understated $\sim$ 1.6x (pure transfer)
Railgun	withdraw (unshield)	1,580,470	1,068,373	1.5x	\$0.80	consistent (sample variance)
Hinkal	deposit (native)	1,048,460	1,076,400	1.0x	\$0.54	consistent
Hinkal	withdraw (native)	538,679	995,149	0.54x	\$0.28	scope differs: measured mixes ERC20 nullifiers, native is cheaper
zERC20	deposit (wrap)	87,814	92,333	1.0x	\$0.10	consistent
zERC20	withdraw (unwrap)	156,590	82,867	1.9x	\$0.08	subgraph cumulative $\sim$ 2x recent; likely historical variance
zERC20	transfer (teleport)	340,917	341,480	1.0x	\$0.18	consistent
Intmax	deposit (native)	200,569	200,600	1.0x	\$0.05	consistent; Informational 2 impact negligible
Fluidkey	transfer	672,425	609,862	1.1x	\$0.36	consistent (measured noisy)
Tornado	deposit	935,701	⚠ RPC-blocked	⚠ n/a	\$0.48	⚠ not measurable — sanctioned, public RPCs refuse the instances
Tornado	withdraw	409,754	⚠ RPC-blocked	⚠ n/a	\$0.21	⚠ not measurable — sanctioned, public RPCs refuse the instances
Curvy (Arb)	transfer	366,717	579,752	0.63x	\$0.01	measured mean skewed by 2 large txs, typical $\sim$ 370k matches
Blanksquare (Base)	deposit	1,751,838	1,785,317	1.0x	\$0.10	consistent
Blanksquare (Base)	withdraw	1,822,645	1,843,899	1.0x	\$0.21	consistent
Veil (Base)	deposit (queued)	360,423	359,539	1.0x	\$0.00	user queue leg — accept leg operator-paid (note)
Veil (Base)	deposit (accepted)	1,357,795	1,363,790	1.0x	leg excluded	operator-paid, excluded by design (note)

Protocol	Operation	Subgraph gas	Measured gas/tx	Δ	Surface d cost	Note
Veil (Base)	withdraw	1,304,709	1,310,879	1.0×	\$0.01	consistent (deduped per tx)
Redact (Sep)	deposit	419,032	412,748	1.0×	testnet	consistent
Redact (Sep)	withdraw	544,182	536,452	1.0×	testnet	consistent
Intmax (Scroll)	withdraw	117,857	779,990	0.15×	\$0.00	basis differs: per-withdrawal vs per-tx, Medium 1 (6.748 ev/tx)

The deployed aggregates settle the value question. Privacy Pools deposit at pool `0xf241d57c` is the one materially wrong number — 4.6M against a real ~410k, while the control pool is correct under the same code. Railgun transfer is understated ~1.6× by its per-output-commitment basis (High 1). Intmax Scroll withdraw is reported per-withdrawal (divided by 6.748 events per transaction) where peers report per-transaction (Medium 1). Veil's surfaced deposit covers the user's queue leg only; the ~1.36M accept transaction is operator-paid and excluded by design (see the Veil note). One row to glance at rather than a finding: zERC20 unwrap's cumulative subgraph average (157k) is ~2× the recent on-chain sample (83k), most likely older unwraps being pricier. Everything else matches the chain within sampling.

The sync- and constant-layer defects do not appear in these aggregates because they are not subgraph numbers: Worm withdraw (High 2), the hardcoded constants (Informational 1), Curvy's cross-network pricing (Low 9), and Monero's raw XMR (Low 1).

Appendix A — panel

Six auditors reviewed the whole codebase independently (indexer specialist, gas/economics analyst, data-pipeline reviewer, backend-TypeScript reviewer, adversarial falsifier, lead), each scoring every confirmed finding. An independent adversarial re-review then re-graded the findings to surfaced-cost impact — reproducing each magnitude against the deployed subgraphs and RPCs — and merged shared root causes. The table maps every original finding to its final ID and severity.

Original	Disposition	Final
Critical 1	uphold, relabel operational	Critical 1
Critical 2	merge → guard finding	High 2
High 1 (21k constant)	downgrade (value correct)	Informational 1
High 2 (Railgun deposit)	merge → Railgun cluster (latent)	High 1
High 3 (Railgun transfer)	uphold (cluster anchor)	High 1
High 4 (Intmax withdraw)	downgrade	Medium 1
High 5 (Monero XMR)	downgrade	Low 1
High 6 (Intmax blended)	downgrade	Informational 2
High 7 (no guard)	uphold (absorbs Critical 2)	High 2
Medium 1 (Hinkal)	downgrade (average survives)	Low 2
Medium 2 (Veil misclass)	downgrade (latent)	Low 3
Medium 3 (Veil endian)	downgrade (value never read)	Informational 3
Medium 4 (PP keying)	downgrade (latent)	Low 4
Medium 5 (Tornado value)	downgrade (non-surfaced)	Low 5
Medium 6 (Railgun count)	merge → Railgun cluster	High 1
Medium 7 (note-merge)	downgrade (notes-only)	Low 6
Low 1 (Redact/Veil value)	downgrade	Informational 4
Low 2 (PP stat order)	uphold	Low 7
Low 3 (eth_call no try_)	uphold	Low 8
Low 4 (dup counter)	downgrade	Informational 5
Low 5 (reload+save)	downgrade	Informational 6
Low 6 (null-receipt)	downgrade	Informational 7
Low 7 (Curvy network)	uphold	Low 9
Informational 1 (integer div)	merge → Railgun cluster	High 1
Informational 2 (Math.trunc)	uphold	Informational 8
Informational 3 (Number())	uphold	Informational 9
Informational 4 (Veil accept leg)	remove → documentation note	—
Inconclusive 1 (Worm dedup)	uphold	Inconclusive 1
Refuted 1 (shared cache)	uphold refutation	Refuted 1

Two clusters collapsed: the Railgun commitment-counting mechanism (original High 2, High 3, Medium 6, Informational 1) into High 1, and the unguarded-NaN run-abort (original Critical 2, High 7) into High 2. The re-grade rule is impact on the surfaced cost — a defect that leaves the published

average correct, of which Hinkal's per-log double-count is the clearest case, drops to Low or Informational however real the bug.

Appendix B — reproduction

`audit/reproduce/reproduce.ts` regenerates this evidence over fixed block ranges, so it returns the same numbers on every run. It queries the deployed subgraph aggregate and measures the on-chain side via `eth_getLogs` + receipts, reading the endpoints from the project `.env` files. Run: `npx tsx audit/reproduce/reproduce.ts`. Full output, including all sampled transactions, is in `audit/reproduce/results.json`.

Five sampled transactions per operation, one per row, full hashes (gas in units), at the blocks recorded in `results.json`:

Protocol	Operation	Transaction	Gas
Privacy Pools	deposit (ETH pool 0xf241)	0x0dff603d9b00a1fe72164c994688964034ce645d39b0a5fc8d69c5e9bf74e276	381,540
Privacy Pools	deposit (ETH pool 0xf241)	0x4eccf73f36977310235b97a4dd1d42fc5375e03b86f02fe3015167b5957ff7b5	397,515
Privacy Pools	deposit (ETH pool 0xf241)	0x4fa8f1e5fbb9df26214b0060ea60acfedcd82fc4bb27d06b06bf700a963ca1ec	381,540
Privacy Pools	deposit (ETH pool 0xf241)	0xea563507a328cbff157284f36ec13fc59df0086e23bfb980e4a83ca0a203f0a	500,875
Privacy Pools	deposit (ETH pool 0xf241)	0x56d0ebe1996863736e926061ae6aed0f7eeef64e09f9f34bc41677d5d2351c72	429,463
Privacy Pools	withdraw	0xe7f18e4fe20ab83936433e0adfb35a785e42b455b56fa83da0e60e5684c6267	600,205
Privacy Pools	withdraw	0x18a4c30a3bc4d4eb3d0c2da6a89516e1cb7343684a1be273e90c2638eae3a3db	616,179
Privacy Pools	withdraw	0xec7f003358f00e7e6266fd9ba6746cdb841dbf012bbfd0354f0e8b3db5ebbc25	588,980
Privacy Pools	withdraw	0x6563eb221270cb8e1a76c1e7dce7ca52727d494b1e05feecb5a41e5c753161b3	626,167
Privacy Pools	withdraw	0x7c29f614e1ddadca53a94ddc68197f3e521552b96d7ee7ed3f9a635597c577fd	616,191
Railgun	deposit (shield)	0x47dd6824dcf6fb17ec63ead906c878ded4ef7492df1ecba50efa6dbd299cb21b	751,506
Railgun	deposit (shield)	0x616df3f981258efa60bd77171863ffb5bbcaa3e579bfc00435c7746e3fd2484a	751,557
Railgun	deposit (shield)	0x927173d5d29ec03981c857c56a508bb7e42566bdd88ea4287656d9af3b4adc50	751,545
Railgun	deposit (shield)	0xe5c36450e68627f99b435043024bf62494207e02655e051246003dccc003be80	775,496
Railgun	deposit (shield)	0x1e08e15a0828e92c6d0e6d9552b63108be63cb04a4a36ef2e3da4fc7da505618	750,979
Railgun	withdraw (unshield)	0xaed6836efc0e536636267f565baf4a973cae227e2ec81715423646bd97ef0d10	1,215,638
Railgun	withdraw (unshield)	0x1f1081b106a533cb21611590fbf29e0e482390218cf03d42c6ead11014291ce0	1,282,930
Railgun	withdraw (unshield)	0x84c9c7edc2861a69cf2c50834755b5d1fe2152ebd5017143b56c8b75bc4f54ee	464,997
Railgun	withdraw (unshield)	0xf33efb9eb3c8e2c52e9f8890e8fd1a2fbdd8990218f5e476f160cd298874677b	1,254,661

Protocol	Operation	Transaction	Gas
Railgun	withdraw (unshield)	0xbb7c425b006c7f33c3c4addabc2ef91190f7fb9f46ceca391496851a6f9eef98	1,123,641
Railgun	transfer (transact, no unshield)	0x2bd1347977d4abd2f77354c6bbeeeb21c6c2df8c20e1a959bd8518979f335fd8	1,141,951
Railgun	transfer (transact, no unshield)	0xe78e8edfffc3ade898a7de40d88a58e7ba57469fb7bdd39ea0ae466287f499a6a	1,080,151
Railgun	transfer (transact, no unshield)	0xa7db4a1c91fbf41d67fa9bf32d38fb9ff91468f1bfff1ca8dc3f591d6d98d26c1	1,097,686
Railgun	transfer (transact, no unshield)	0xeb76620b16e72ba5f341be30ef585363294bcb80735ca0ed30e973d85a90dfcc	1,012,306
Railgun	transfer (transact, no unshield)	0x561d033d4d9a0adcf4977a904f4e586090723d098453f05c3e8753d6302bf3c2	981,783
Hinkal	deposit/transfer (NewCommitment)	0x60a4b5921775479f6d8f35c8ab9858d56aa86b88408487d844dcd83c93e38d11	1,102,763
Hinkal	deposit/transfer (NewCommitment)	0x57216b2a81b805e05cbd8b2acb695fc7f6dd60c2ebcca79c44c1ab8650b58a38	1,057,822
Hinkal	deposit/transfer (NewCommitment)	0xe4f28ae935e478c6de1802efe825e21ec1914d9eca7007e3acd6bf7522cc470d	1,109,844
Hinkal	deposit/transfer (NewCommitment)	0xdecb4bc7df0c2b370cad0693c9bc0934749f642f5b0c350195ca75d0fdf8e6e	1,116,368
Hinkal	deposit/transfer (NewCommitment)	0xd330e2378704fa7289b6f05411c9f36db57bfbbbe38fdc794bd5720c990fa037	995,201
Hinkal	withdraw (Nullified)	0x0700d89072084fcbf54ac0c47c9e0b8ccd2dba03c45c836c1d8d9b3100fe730d	1,239,546
Hinkal	withdraw (Nullified)	0x60a4b5921775479f6d8f35c8ab9858d56aa86b88408487d844dcd83c93e38d11	1,102,763
Hinkal	withdraw (Nullified)	0xe4f28ae935e478c6de1802efe825e21ec1914d9eca7007e3acd6bf7522cc470d	1,109,844
Hinkal	withdraw (Nullified)	0x3e0eb0b55375f2b42a74f0f5f2b27b55a95d95b3df7adae2b1046165f494c583	528,391
Hinkal	withdraw (Nullified)	0xd330e2378704fa7289b6f05411c9f36db57bfbbbe38fdc794bd5720c990fa037	995,201
zERC20	deposit (wrap)	0x8fd4f4a12b5115529177f9f9061a96bb5766aa112d314d38307d8095ed29b519	78,646
zERC20	deposit (wrap)	0x6567327d9e73acd248fa11c0db52ce1b79afd813318e06bdc7c1fa5681dc2787	95,758
zERC20	deposit (wrap)	0xe415fc32642e821172139e8d0431d257b6b9b00a626e1b004898a6763c0160c4	95,758

Protocol	Operation	Transaction	Gas
zERC20	deposit (wrap)	0xc3b2ff2ac39002ab12709c4508fe8bcd701cd8e27a54b8857a5663378efc33e4	95,758
zERC20	deposit (wrap)	0xbe8cce280998e6b0ea8bd29490b1bbf2756da0f5610f1aceb1e0e5ae1cb94b4e	95,746
zERC20	withdraw (unwrap)	0x4cae1f1a8282b3ac103f5fd2cf1477536cf6df977d8f3cbd65c5d2ea1f6970b4	85,723
zERC20	withdraw (unwrap)	0x98cf510e5f848de16b90f2652210a8c15fda624a16c73074a3445eca8f8911f1	85,723
zERC20	withdraw (unwrap)	0x66b7b55227e71695d87f4cfc7a6d55dc5b85012d16a4bcc246cda82e48d406d	80,971
zERC20	withdraw (unwrap)	0x4354cfcea87ff09e6c9ec3693d0775ac704d160c39c8e179a2b1c1edbe32269a	80,971
zERC20	withdraw (unwrap)	0x2454ec5488925cf5d33d67092499846826e70dbd065a1517d4dee33dc0f3e015	80,947
zERC20	transfer (teleport)	0xdd277e38baa8db473d6e32e81bb018a78398c1d283db65ba5cb4fad0fe79e6f9	338,038
zERC20	transfer (teleport)	0xb20e25f6831e09b224a13b5e8ed1843c5f36aa55ee0442f94861b4dcce03f4ca	338,050
zERC20	transfer (teleport)	0xfd38fc6f3336d51a261f2d714223d106be1db5b0eb8d634743ced770ae7c7df0	338,074
zERC20	transfer (teleport)	0x26fe1073cfd38f3d1557e67ae3d8e37ebe65a10a1e6ad9066ed3209bb32e15e2	338,062
zERC20	transfer (teleport)	0xaf50d7435c2e288e45c77f10657245626ed04ea961d1223e7032a5151910e07e	355,174
Intmax	deposit	0x82d81f260c4fb1c34385c0b448969251470a3507ea5a19d98dea07b2364d04cc	200,593
Intmax	deposit	0xff1a1fa672a9e12f2b447b4133a2ae5e206f470b34f6d57b9c799338f5e7bfc3	200,605
Intmax	deposit	0xac27530d39630b6f5b6cee69d0c5f58f38c6bf7e7ed5ab40baab93dfe7697127	200,605
Intmax	deposit	0x95ac3f3ea599c2c59920d1c8612426d5ef22e3a2947a743a028c721c21b08458	200,605
Intmax	deposit	0x6de98a249147cb1b384add0a9e1770094728fffb811f9909ba0c59c528e4d37	200,593
Fluidkey	transfer (OperationRelayed)	0x05971b031d8dac44b055911825cff5031621fd84e39855f4cde6e9eca666897a	1,614,761
Fluidkey	transfer (OperationRelayed)	0x65ef61808f0fd9d7bb65f09997f6d929884f1b7cfb1c614e97a6f6681be3629f	111,114

Protocol	Operation	Transaction	Gas
Fluidkey	transfer (OperationRelayed)	0x74fe67499ce3b7d63f8621793fcccc98bd3bd30491038cea42bc16096e6553be	389,968
Fluidkey	transfer (OperationRelayed)	0x49063a2af5dec8be0dd9b553b898b211d63f786046440df902c16f140fcab47a	494,603
Fluidkey	transfer (OperationRelayed)	0xb0f3d0cc76bcd02a8cb48631a03697cca6c61438e3c0658319582528e0be2bfe	438,866
Curvy (Arb)	transfer (Withdraw)	0x07de83cfc340a26efad9ab1489e49381fbccbf3202917b2df7ba0f90cc2452	361,947
Curvy (Arb)	transfer (Withdraw)	0xc5dff864a51316fd68676ebd8e826f95818de78c5ed11e5dc8ea5678f0f4400	370,404
Curvy (Arb)	transfer (Withdraw)	0x4d92b38bbb345eb714dbc8f79dedf46ac2866f66df4bbcca272f5d72b8e97436	337,993
Curvy (Arb)	transfer (Withdraw)	0x4163e08caacc6fc1d87641343897e4048e3274503fc430d5ad0f63a22d6091ca	910,585
Curvy (Arb)	transfer (Withdraw)	0xf15fc292d9f5bc024eb5e8d777cebf0e77c809c8f9e2a3250989f5c79566dbc	917,832
Blanksquare (Base)	deposit	0xea7debe1805f26e9bb32e101fba950e88bcece7513b0bebd8766038c5b8ff943	1,778,515
Blanksquare (Base)	deposit	0x9c2bd93dde2811224b1ac4a7423e97bd426ede7ccd9c117cbe19c686b15fd28b	1,778,527
Blanksquare (Base)	deposit	0x4fd13bd9e996fbb8a8a0825341b9dde2238fe456ea87329df57abac31f5629477	1,778,503
Blanksquare (Base)	deposit	0x214dd6fd59e2d56d6a6c30a45729511ba9908e305b7d8a972c6d613f88444d89	1,778,371
Blanksquare (Base)	deposit	0xae0fb253fd8699b627321a321a818fdccdfac2e99109515f63d1c02694a0c41	1,812,667
Blanksquare (Base)	withdraw	0x5b072e91fc46980c1ebe16a54ac0dddfccd99475e00481b363a3865a75ca5201	1,837,403
Blanksquare (Base)	withdraw	0x76bedeaabad1bb05dff4601d0aea4a68d69e163a98938fa2096d2aa0a58f750f	1,837,415
Blanksquare (Base)	withdraw	0x4e560416c3f4756fe9896742f619409e862e21a0b5e7ae5c72b931a97327bad2	1,889,459
Blanksquare (Base)	withdraw	0x0d0da214123842914ed2a1521b21e0772994f5c9d66492dc352eeafa2e5e6ed3	1,817,961
Blanksquare (Base)	withdraw	0x2622d53c44f702398c8ea64a29e09764f217a7728802dc2a22990395645bc61e	1,837,259
Veil Cash (Base)	deposit (queued)	0x98ff045d010d3b83772dbd51a5731808f4e9e44108aaae0040b01f93ca7a042f	373,505

Protocol	Operation	Transaction	Gas
Veil Cash (Base)	deposit (queued)	0x7a64d0612f944fb407a11b1d4d725beeb1976a94e27b272749cd0f0fb9d023f6	373,517
Veil Cash (Base)	deposit (queued)	0x647498e734e6aed1be1b50a659b17504d16561c51fdfd0e26d20dc999cd41006	373,505
Veil Cash (Base)	deposit (queued)	0x85eb48ff3f8363699dc855fbef0cf5294a52985d352b17e86eeb33282ee771b2	303,652
Veil Cash (Base)	deposit (queued)	0xd746a502957bcaa3d4c0e3f2aa618b713abb1f6edc684956404a1ae00afc8791	373,517
Veil Cash (Base)	deposit (accepted)	0x27328558327409c7fd663857c208c51c8cb5e58dc0c9d04edcaf3679ddf54140	1,361,908
Veil Cash (Base)	deposit (accepted)	0x24a0a671f1521b0b8ea4dc9c88c055c0be0ce771633f04fc1f665dfd018bd098	1,361,993
Veil Cash (Base)	deposit (accepted)	0xc3ec4bbb1a45e842baa820aaf8a31310ccd24f2f281ad5b4c52b48220aab3744	1,358,982
Veil Cash (Base)	deposit (accepted)	0xeeee352882909925e0f1a0df5d0c77d9a64992be871bb67c34aa9bf9693f6c228	1,368,034
Veil Cash (Base)	deposit (accepted)	0xe6c225dc3a1320549ebe0f574347cdfd19d3f25016f4d17b585cfe0bc87e0068	1,368,035
Veil Cash (Base)	withdraw/transfer (NewNullifier)	0x6ecef35c6f90d68653aff5b2560f036d7fa1f379b69471d71149eb6010f496d	1,276,513
Veil Cash (Base)	withdraw/transfer (NewNullifier)	0xeeee352882909925e0f1a0df5d0c77d9a64992be871bb67c34aa9bf9693f6c228	1,368,034
Veil Cash (Base)	withdraw/transfer (NewNullifier)	0xa781964a590e5a5fb12b423e0ddbdbf8af41aae571b394caa43b82d8cb037121	1,270,895
Veil Cash (Base)	withdraw/transfer (NewNullifier)	0xe6c225dc3a1320549ebe0f574347cdfd19d3f25016f4d17b585cfe0bc87e0068	1,368,035
Veil Cash (Base)	withdraw/transfer (NewNullifier)	0x37a0f6aa4e07375612598c957c85b37d6a0ca28774e66ef0d76cc9a6eac02012	1,270,920
Redact (Sepolia)	deposit (ShieldedNative)	0x96e497f61f2073d1c3a41454bb96a70cadb3a4f9874bd75fd9f92ef25b389136	403,294
Redact (Sepolia)	deposit (ShieldedNative)	0x3d3289302886c71db389b312bace900eb512815606a2445e0fc2a220b79c1089	403,294
Redact (Sepolia)	deposit (ShieldedNative)	0xdceec0d59b3f90b2b6928ad0d8e670f7b8afb5ff35f35c29e105be64dd98ea2fb	429,328
Redact (Sepolia)	deposit (ShieldedNative)	0xc9feca3a41661ff1684a4ae69fbb8bb8169e385aea2b538beeedd06f217b4620	403,294
Redact (Sepolia)	deposit (ShieldedNative)	0xdf2775aa39bfdfe787312211338246228e1aa0d1027aa17d332a7c1c9d631db7	424,528

Protocol	Operation	Transaction	Gas
Redact (Sepolia)	withdraw (Unshielded)	<code>0x436eb808533362549b923ce2c94cb07dec3a16767d13d293bec17bd5b6d23a2b</code>	536,450
Redact (Sepolia)	withdraw (Unshielded)	<code>0xdcc49e91c34cb29601fdc6a5573415fa6d074ffbcb8f631767ecbff1b3e2e942</code>	536,462
Redact (Sepolia)	withdraw (Unshielded)	<code>0xc1458629be3a2c7b783603949ebda353b2c8e4dbe8205323967592aa1f0080bf</code>	536,462
Redact (Sepolia)	withdraw (Unshielded)	<code>0xbd9743ee1f704008235f02d9b635c4f4b0de29ea1d5a3e2ad5104314f9b9225e</code>	536,462
Redact (Sepolia)	withdraw (Unshielded)	<code>0x1c540cfe78dd424ae656941adcd20d9811c8e8b3092e2c71e612590c19ebcfaa</code>	536,426
Intmax (Scroll)	withdraw (DirectWithdrawalQueue d)	<code>0x44ec1bc2cb50d592afb2b701a832309d5f24409fcd00b6c0958f9b3c27190370</code>	792,088
Intmax (Scroll)	withdraw (DirectWithdrawalQueue d)	<code>0x7613bc4349afc1946cb124ea4c64b295951c101d87c808f9b1ae3950268a3747</code>	725,524
Intmax (Scroll)	withdraw (DirectWithdrawalQueue d)	<code>0x6090fdf2471d0b07cfa32132ce3c2390a823d8d4a75315aae044904ce1a3d5fb</code>	720,515
Intmax (Scroll)	withdraw (DirectWithdrawalQueue d)	<code>0x3dcef650806d7b3f87720e5067ff94f34b1d67eb5c99868a9d0ec301952522b1</code>	717,554
Intmax (Scroll)	withdraw (DirectWithdrawalQueue d)	<code>0xa22dc5bf9b9a028e319bb281c3a81fefe870177820505d44e69c9f8267c1856b</code>	944,268

Tornado Cash (sanctioned, RPC-blocked), Worm (call handler, no event), and Monero (non-EVM) have no on-chain sample here.

Appendix C — references

- The Graph — AssemblyScript API, handlers and receipts, best practices: <https://thegraph.com/docs/en/subgraphs/best-practices/>
- Trail of Bits `property-based-testing` and `differential-review` skills: <https://github.com/trailofbits/skills>
- Reference report structures consulted: ZKSecurity (intmax-zerc20, hinkal, aptos, tongo), Trail of Bits, Zellic published audits (see `audit/references/`).